

Table des matières

Spec — Module Bitfocus Companion : Crestron DM NVX	1
1. Objectif et périmètre	1
2. Architecture des fichiers	2
3. Authentification (réécriture complète)	2
4. Client HTTP (api.ts — corrections)	4
5. Mapping endpoints NVX	4
6. Détection du mode encodeur/décodeur	5
7. Stratégie de connexion et retry	6
8. Actions	6
9. Feedbacks	7
10. Variables	8
11. Gate hardware — plan de validation	8
12. WebSocket / XioSubscription (point ouvert)	9
13. Packaging open source	9
14. Plan d'implémentation (4 phases)	11
15. Questions ouvertes	12

Spec — Module Bitfocus Companion : Crestron DM NVX

Date : 2026-05-18

Version cible : 1.0.0

Auteur : Didier Casalta

Statut : Approuvé — prêt pour implémentation

1. Objectif et périmètre

But

Créer un module Bitfocus Companion (v4.x) pour contrôler les appareils Crestron DM NVX (encodeurs/décodeurs AV-over-IP) via leur API REST, en vue d'une publication officielle sur le dépôt Bitfocus (companion-module-requests).

Périmètre v1.0

Domaine	Inclus
Stream AV	Routing, mode enc/dec, nom, multicast, URL, enable/disable
Audio	Volume absolu/relatif, mute/unmute/toggle
Vidéo	Sélection entrée HDMI, signaux présents
Appareil	Reboot, info (nom, firmware, IP)
Réseau	Lecture seule (IP, mode DHCP) via /Device/Ethernet
Hardware	Température, uptime, MAC via /Device/DeviceSpecific

Domaine	Inclus
Ports	Lecture état ControlPorts (RS-232, IR) via /Device/ControlPorts
Discovery	Connexion par nom de stream via /Device/DiscoveryConfig
WebSocket	Architecture préparée — implémentation v1.1 (voir §9)

Hors périmètre v1.0

- Écriture config réseau (risque en production)
- XiO Cloud / gestion multi-appareils centralisée
- Scheduling / événements programmés

2. Architecture des fichiers

Structure inchangée par rapport au code existant — elle est correcte.

```
companion-module-crestron-nvx/
├── companion/
│   ├── manifest.json    Métadonnées format v4 (réécriture complète)
│   └── HELP.md          Guide utilisateur (à créer)
├── src/
│   ├── index.ts         Classe principale, lifecycle, polling
│   ├── config.ts        Champs de configuration UI (étendu)
│   ├── api.ts           Client HTTP NVX (réécriture complète)
│   ├── actions.ts       Définitions des actions (mise à jour)
│   ├── feedbacks.ts     Définitions des feedbacks (+ subscribe/unsubscribe)
│   ├── variables.ts     Définitions des variables (étendu)
│   └── presets.ts       Presets prêts à l'emploi (mise à jour)
├── docs/
│   └── superpowers/specs/ Spec et plans
├── README.md            (à créer)
├── CHANGELOG.md         (à créer)
├── LICENSE              Fichier texte MIT (à créer)
├── .gitignore           (à créer)
├── eslint.config.mjs    Config ESLint avec generateEslintConfig() (à créer)
├── package.json         Mise à jour dépendances
└── tsconfig.json        Strict mode confirmé
```

3. Authentification (réécriture complète)

Problèmes du code actuel

Code actuel	Réalité documentée
POST /userlogin	POST /userlogin.html (endpoint incorrect)
1 étape	2 étapes obligatoires
Cherche SessionToken JSON	Pas de token JSON — cookies uniquement
Ignore Set-Cookie	5+ cookies à capturer dans les headers de réponse
Gère 401 → re-login	La session expirée retourne 403, pas 401
Pas de logout	GET /logout à appeler dans destroy()
AuthByPasswd ignoré	Change à chaque réponse — doit être relu et renvoyé

Flux d'authentification correct (3 étapes)

Étape 1 – Amorçage

```
GET /userlogin.html
← Set-Cookie: TRACKID=<valeur>
```

Étape 2 – Login

```
POST /userlogin.html
Cookie: TRACKID=<valeur>
Body: { "login": "<user>", "passwd": "<pass>" }
← Set-Cookie: TRACKID, userstr, userid, iv, tag    (constants session)
← Set-Cookie: AuthByPasswd=<valeur_initiale>      (change à chaque réponse)
```

Étape 3 – Requêtes (toutes)

```
<METHOD> /Device/...
Cookie: TRACKID=...; userstr=...; userid=...; iv=...; tag=...; AuthByPasswd=<courante>
← Set-Cookie: AuthByPasswd=<nouvelle_valeur>      → mettre à jour le cookie jar
```

Cookie jar (**api.ts**)

```
private cookieJar: Map<string, string> = new Map()

// Extrait tous les Set-Cookie d'une réponse et met à jour le jar
private extractCookies(rawHeaders: string[]): void

// Construit le header Cookie à envoyer
private buildCookieHeader(): string

// Vide le jar (déconnexion, changement de config)
private clearCookies(): void
```

Capture des headers dans **nodeFetch**

nodeFetch doit retourner les headers de réponse en plus du body :

```
// Type de retour étendu
{ status: number; ok: boolean; body: string; headers: Record<string, string | string[]> }
```

```
// Dans le callback res.on('end')
res.headers['set-cookie'] // tableau de strings
```

Gestion de session

- 403 → re-login automatique + retry une fois (remplace le 401 actuel)
- Timeout de session non documenté → détecté par le 403, pas de refresh proactif
- Logout : GET /logout appelé dans destroy() pour libérer la session proprement

Point ouvert — requêtes parallèles

AuthByPasswd changeant à chaque réponse, les requêtes Promise.all simultanées créent un état de course sur le cookie jar. Décision différée à la gate hardware : tester si le NVX tolère plusieurs requêtes avec le même AuthByPasswd ou si les requêtes doivent être séquentielles. Le code sera structuré pour permettre les deux sans réécriture majeure (stratégie requestQueue optionnelle).

4. Client HTTP (api.ts — corrections)

Problèmes à corriger

Problème	Correction
nodeFetch ignore Set-Cookie	Capturer res.headers['set-cookie']
Pas de timeout HTTP	req.setTimeout(10_000, () => req.destroy())
403 non géré	Ajouter 403 → re-login + retry (comme le 401 actuel)
catch(err) { throw err } dans getDeviceStatus	Supprimer le try/catch inutile
getStatus() placeholder retourne undefined	Supprimer, utiliser directement InstanceStatus

Timeouts

- Timeout par requête HTTP : 10 secondes
- Timeout de connexion TCP : 5 secondes (option timeout dans https.request)

5. Mapping endpoints NVX

Corrections obligatoires

Usage	Endpoint actuel ☐	Endpoint correct ✓
Config encodeur (stream émis)	/Device/AvSignal	/Device/StreamTransmit
Config décodeur (stream reçu)	/Device/AvSignal	/Device/StreamReceive
Audio + sélection HDMI	/Device/AudioControl + /Device/VideoSwitch	/Device/AudioVideoInputOutput
Info appareil	/Device/DeviceInfo	/Device/DeviceInfo ✓
Reboot	/Device/DeviceOperations	/Device/DeviceOperations ✓

Nouveaux endpoints à couvrir

Endpoint	Usage
/Device/DeviceSpecific	Température, uptime, MAC (variables monitoring)
/Device/Ethernet	IP courante, mode DHCP/statique (lecture seule)
/Device/ControlPorts	État RS-232, IR (lecture)
/Device/DiscoveryConfig	Discovery par nom de stream pour routing
GET /logout	Déconnexion propre dans destroy()

Propriétés JSON exactes

Non confirmées — les noms de propriétés dans StreamTransmit, StreamReceive, AudioVideoInputOutput etc. doivent être validés sur un appareil réel lors de la gate hardware (§11). Le code de phase 1 utilisera des constantes nommées avec des TODO :

```
// TODO: valider le nom exact lors de la gate hardware
const PROP_STREAM_URL = 'StreamUrl' // hypothèse à confirmer
```

6. Détection du mode encodeur/décodeur

Comportement cible

Mode détecté	Polling	Actions disponibles
Encoder	StreamTransmit + AudioVideoInputOutput + DeviceInfo	Nom stream, multicast, enable/disable encodeur, HDMI in, audio
Decoder	StreamReceive + AudioVideoInputOutput + DeviceInfo	URL source, discovery par nom, HDMI out, audio
Both	StreamTransmit + StreamReceive + AudioVideoInputOutput + DeviceInfo	Toutes les actions

Détection automatique

Au démarrage, le module interroge les deux endpoints : - Si StreamTransmit répond avec des données valides et StreamReceive aussi → mode Both - Si seulement StreamTransmit → mode Encoder - Si seulement StreamReceive → mode Decoder

Fallback configuration manuelle

Champ deviceMode dans config.ts :

Auto (détection) | Encoder only | Decoder only | Both

Si Auto échoue ou retourne un résultat ambigu, le module utilise la valeur manuelle.

7. Stratégie de connexion et retry

Configuration (`config.ts` — nouveaux champs)

Champ	Type	Défaut	Description
<code>retryMode</code>	dropdown	<code>exponential</code>	<code>exponential</code> ou <code>fixed</code>
<code>retryInterval</code>	number	<code>10000</code>	Intervalle fixe en ms (visible si <code>retryMode = fixed</code>)

Retry exponentiel

10s → 20s → 40s → 80s (cap à 80s, reset au reconnect réussi)

Retry fixe

Intervalle constant configurable par l'utilisateur (min 5s, max 300s).

Statut Companion

Le module utilise le statut natif de l'instance Companion : - `InstanceStatus.Connecting` — tentative en cours - `InstanceStatus.Ok` — connecté et polling actif - `InstanceStatus.ConnectionFailure` — retry en cours (message avec délai prochain essai) - `InstanceStatus.Disconnected` — `destroy()` appelé

8. Actions

Actions existantes — corrections de routing

ID action	Changement
<code>set_stream_url</code>	Cible <code>StreamReceive</code> (décodeur)
<code>set_stream_name</code>	Cible <code>StreamTransmit</code> (encodeur)
<code>set_multicast_address</code>	Cible <code>StreamTransmit</code> (encodeur)
<code>set_video_input</code>	Cible <code>AudioVideoInputOutput</code>
<code>mute_audio / unmute_audio / toggle_audio_mute</code>	Cible <code>AudioVideoInputOutput</code>
<code>set_audio_volume / adjust_audio_volume</code>	Cible <code>AudioVideoInputOutput</code>
<code>reboot_device</code>	Inchangé ✓
<code>set_stream_mode</code>	Inchangé (cible <code>DeviceOperations</code> ou <code>StreamTransmit/Receive</code>)

Nouvelles actions

ID action	Description
<code>enable_stream</code>	Active la transmission (encodeur)
<code>disable_stream</code>	Désactive la transmission (encodeur)

ID action	Description
connect_to_stream	Connexion décodeur par nom de stream (discovery via DiscoveryConfig)

Action `connect_to_stream` — design

```
connect_to_stream: {
  name: 'Connect to Stream by Name',
  description: 'Discover encoder by stream name and connect decoder',
  options: [
    { type: 'textinput', id: 'name', label: 'Stream Name', useVariables: true }
  ],
  callback: async (action, context) => {
    const name = await context.parseVariablesInString(action.options.name as string)
    const url = await api.discoverStreamUrl(name) // GET /Device/DiscoveryConfig
    if (url) await api.setVideoSource(url)
    else this.log('warn', `Stream "${name}" not found via discovery`)
  }
}
```

9. Feedbacks

Ajout **subscribe** / **unsubscribe** sur tous les feedbacks

Non obligatoire fonctionnellement pour v1.0 (polling), mais requis pour les bonnes pratiques Companion et prépare l'architecture WebSocket :

```
subscribe: async (_feedback) => {
  // no-op en mode polling
  // en mode WebSocket (v1.1) : s'abonner à l'événement correspondant
},
unsubscribe: async (_feedback) => {
  // no-op en mode polling
  // en mode WebSocket (v1.1) : se désabonner
},
```

checkFeedbacks sélectif

Remplacer `this.checkFeedbacks()` global par des appels ciblés selon ce qui a changé :

```
if (prev.audioMuted !== next.audioMuted) this.checkFeedbacks('audio_muted')
if (prev.streamMode !== next.streamMode) this.checkFeedbacks('is_encoder', 'is_decoder')
// etc.
```

Nouveaux feedbacks

ID	Description
stream_enabled	Actif quand la transmission encodeur est activée
device_mode_both	Actif quand le device est en mode Both

10. Variables

Variables existantes (inchangées)

stream_mode, stream_url, stream_name, multicast_address, video_source, video_source_name, audio_muted, audio_volume, hdmi_input_signal, hdmi_output_signal, device_name, firmware_version, ip_address, connection_status

Nouvelles variables

Variable ID	Source	Description
encoder_url	StreamTransmit	URL RTSP/multicast de l'encodeur local (pour routing cross-device)
stream_enabled	StreamTransmit	true / false
device_mode	détection	encoder / decoder / both
mac_address	DeviceSpecific	Adresse MAC
device_temperature	DeviceSpecific	Température en °C (si disponible)
device_uptime	DeviceSpecific	Uptime en secondes (si disponible)
network_ip	Ethernet	IP actuelle (lecture seule)
network_dhcp	Ethernet	true / false

11. Gate hardware — plan de validation

Avant la phase 2, tester sur l'appareil réel et documenter dans docs/hardware-validation.md :

Auth

- ☐ Confirmer que GET /userlogin.html retourne bien TRACKID
- ☐ Confirmer que POST /userlogin.html retourne les 5 cookies
- ☐ Vérifier que AuthByPasswd change bien à chaque réponse
- ☐ Tester la durée de session (laisser expirer, observer le 403)
- ☐ Tester GET /logout

Endpoints et propriétés JSON

- ☐ GET /Device/StreamTransmit → capturer la réponse JSON complète
- ☐ GET /Device/StreamReceive → capturer la réponse JSON complète

- ☐ GET /Device/AudioVideoInputOutput → capturer la réponse JSON complète
- ☐ GET /Device/DeviceInfo → vérifier les chemins utilisés
- ☐ GET /Device/DeviceSpecific → capturer la réponse
- ☐ GET /Device/Ethernet → capturer la réponse
- ☐ GET /Device/ControlPorts → capturer la réponse
- ☐ GET /Device/DiscoveryConfig → vérifier le format de discovery

Comportement parallélisme

- ☐ Tester 2 requêtes simultanées avec le même AuthByPasswd
 - Si acceptées → Promise.all possible
 - Si rejetées → implémentation de la queue séquentielle

Actions

- ☐ Tester POST StreamTransmit avec nom de stream
 - ☐ Tester POST StreamReceive avec URL source
 - ☐ Tester POST AudioVideoInputOutput pour volume et mute
 - ☐ Tester reboot et observer le comportement de reconnexion
-

12. WebSocket / XioSubscription (point ouvert)

Statut

L'API XioSubscription est documentée et non expérimentale. L'implémentation est différée à v1.1 pour ne pas bloquer la publication v1.0.

Architecture préparée

Le code est structuré pour permettre l'ajout du WebSocket sans réécriture : - api.ts expose une interface IConnectionStrategy (polling ou WebSocket) - En v1.0 : seule l'implémentation polling est fournie - En v1.1 : l'implémentation WebSocket implémente la même interface

Question ouverte à trancher en v1.1

Comment polling et WebSocket coexistent-ils ? Options : - A) Choix utilisateur dans la config (WebSocket on/off) - B) WebSocket principal + fallback auto sur polling si indisponible

Décision différée : à évaluer après implémentation et tests WebSocket sur hardware.

13. Packaging open source

manifest.json — format v4 (réécriture)

```
{
  "id": "crestron-nvx",
  "name": "Crestron DM NVX",
  "shortname": "NVX",
```

```

"manufacturer": "Crestron",
"products": [{ "id": "nvx", "name": "DM NVX Series" }],
"description": "Control Crestron DM NVX AV-over-IP encoders and decoders via REST API.",
"version": "1.0.0",
"license": "MIT",
"repository": "https://github.com/bitfocus/companion-module-crestron-nvx",
"bugs": "https://github.com/bitfocus/companion-module-crestron-nvx/issues",
"maintainers": [{ "name": "Didier Casalta", "email": "didier.casalta@gmail.com" }],
"runtime": {
  "type": "node22",
  "api": "nodejs-ipc",
  "apiVersion": "0.0.0",
  "entrypoint": "dist/index.js"
}
}

```

Champs manquants dans le manifest actuel : manufacturer, products (mauvais format), maintainers, format runtime (objet).

package.json — mises à jour

Package	Actuel	Cible
@companion-module/base	^1.8.0	^1.14.1
@companion-module/tools	absent	^2.7.2 (obligatoire)
prettier	absent	^3.0.0
eslint	absent	^9.0.0
@types/node	absent	^22.0.0
engines.node	>=18	"^22.20 \\ \\ ^18.20"

Script format à ajouter : "prettier --write src/**/*.ts"

tsconfig.json

Vérifier que "strict": true est présent. Sinon, l'ajouter.

Fichiers à créer

Fichier	Description
README.md	Installation, configuration, fonctionnalités, troubleshooting
companion/HELP.md	Guide utilisateur : actions, feedbacks, variables, setup
LICENSE	Texte MIT complet
CHANGELOG.md	Section ## [1.0.0] - 2026-... à compléter à la livraison
.gitignore	node_modules/, dist/
eslint.config.mjs	import { generateEslintConfig } from '@companion-module/tools/eslint'

upgradeScripts dans index.ts

Tableau vide à déclarer dès la v1.0, prêt pour les futures migrations :

```
getUpgradeScripts() {  
  return []  
}
```

Processus de soumission Bitfocus

1. Créer un repo GitHub `companion-module-crestron-nvx`
 2. Ouvrir une issue sur `bitfocus/companion-module-requests`
 3. Après accord, pusher le code + tag `v1.0.0`
 4. Déclencher le workflow GitHub Actions → intégration dans les beta builds (<6h)
 5. Contacter Bitfocus sur Slack `#module-development` si besoin
-

14. Plan d'implémentation (4 phases)

Phase 1 — Couche auth + endpoints corrects

Objectif : que le module puisse se connecter et lire des données sur un vrai appareil.

- Réécriture complète de `api.ts` :
 - `nodeFetch` avec capture des headers de réponse
 - Cookie jar (5 cookies + `AuthByPasswd` roulant)
 - Authentification 2 étapes (GET + POST `/userlogin.html`)
 - Gestion 403 → re-login + retry
 - Timeout HTTP (10s par requête)
 - Logout dans `destroy()`
- Correction des endpoints (`StreamTransmit`, `StreamReceive`, `AudioVideoInputOutput`)
- Correction du `getStatus()` placeholder dans `index.ts`
- Propriétés JSON avec constantes TODO (noms à valider)

Critère de fin : le module peut se connecter, s'authentifier et lire `DeviceInfo`.

▼ GATE HARDWARE ▼

Validation sur appareil réel selon checklist §11.

Complétion du fichier `docs/hardware-validation.md` avec toutes les réponses JSON capturées.

Phase 2 — Corrections post-test + SDK Companion complet

Objectif : corriger les chemins JSON validés sur hardware, compléter les bonnes pratiques SDK.

- Remplacer les constantes TODO par les vrais noms de propriétés
- Décision sur le parallélisme (suite aux tests)
- Implémentation du mode `encoder/decoder/both` (détection auto + fallback)
- Nouvelles actions : `enable_stream`, `disable_stream`, `connect_to_stream`

- subscribe/unsubscribe sur tous les feedbacks
- checkFeedbacks sélectif
- upgradeScripts (tableau vide)
- Stratégie retry configurable (exponentiel / fixe)

Phase 3 — Exhaustivité NVX

Objectif : couvrir l'ensemble du périmètre défini.

- Variables étendues : encoder_url, device_mode, monitoring hardware, réseau
- Nouvelles actions et feedbacks liés au mode
- connect_to_stream via /Device/DiscoveryConfig
- Lecture /Device/Ethernet, /Device/DeviceSpecific, /Device/ControlPorts
- Mise à jour des presets pour les nouveaux cas d'usage

Phase 4 — Packaging open source

Objectif : module prêt pour soumission Bitfocus.

- manifest.json format v4
- package.json mise à jour dépendances
- tsconfig.json vérification strict
- eslint.config.mjs avec helper officiel
- README.md, companion/HELP.md, LICENSE, CHANGELOG.md, .gitignore
- Build propre (npm run build sans erreurs)
- Lint propre (npm run lint sans warnings)
- Soumission via companion-module-requests

15. Questions ouvertes

Question	Impact	Décision attendue
Propriétés JSON exactes (StreamTransmit, etc.)	Bloquant pour les actions	Gate hardware (phase 1→2)
Parallélisme requêtes avec AuthByPasswd	Performance polling	Gate hardware
Coexistence WebSocket + polling	Architecture v1.1	Lors de l'implémentation WebSocket
Durée de vie session NVX	Stratégie refresh	Gate hardware
Modèles NVX-363 et D30 — spécificités	Variables conditionnelles	À documenter sur hardware